

✅ การดำเนินการจาก Feedback

Feedback	สถานะ	การดำเนินการ
1. Test cases น้อย	✅	จาก 2 ไฟล์ → เพิ่มเป็น 20 ไฟล์ เพิ่มไป 958 บรรทัดของ test code ซึ่ง 135 tests passing
2. Test หลัง dev	⚠️	ยังไม่ใช่ TDD แต่ได้เพิ่ม tests ที่ครอบคลุม feature ที่พัฒนาไปแล้ว และทีมยังไม่คุ้นเคยกับ TDD workflow จึงยังเป็น "Dev ก่อน → Test ที่หลัง" นอกจากนั้นยังต้องการโครงสร้าง feature พื้นฐานก่อน
3. ไม่ครอบคลุม	⚠️	- Happy path: ครอบคลุมดี (auth, CRUD, component rendering) - Error responses: ครอบคลุมบางส่วน (400, 500 errors) - Edge cases: ยังจำกัด (network failures, empty states, etc.)
4. ไม่มี GitHub Actions	✅	สร้าง .github/workflows/test.yml แล้ว
5. ไม่มี Coverage Threshold	✅	เพิ่ม thresholds 70% ใน vitest.config.ts

ถ้าจะทำใหม่ จะเปลี่ยนวิธีทำงานของตนเอง 1 อย่างคืออะไร?

ณัฐกฤตา รัตนันท์ – หากมีโอกาสได้ทำงานนี้อีกครั้ง จะวางแผนงานให้ชัดเจนขึ้น โดยกำหนด/แบ่งงานตามหน้าที่ของแต่ละคนให้เหมาะสม แก้ปัญหาให้เร็วขึ้นโดยไม่ปล่อยค้างเกิน Sprint และเพิ่มการสื่อสารภายในทีมให้มากขึ้น เพื่อให้การทำงานคล่องตัวและเข้าใจกันมากขึ้น

พรทเวา จินดาธรรม – จะปรับเปลี่ยนวิธีการทำ Sprint Planning ให้รัดกุมมากขึ้น จากเดิมที่การประเมินเวลาอาจจะอิงตามแผนงานรวมเป็นหลัก คราวหน้าจะดึง Developer เข้ามามีส่วนร่วมในการประเมินความเสี่ยงและความซับซ้อนทางเทคนิค (Technical Risk) ของแต่ละ Task ให้ละเอียดตั้งแต่เนิ่นๆ ก่อนที่จะตกลงตั้งพีเจอรันนั้นเข้าสู่ Sprint เพื่อป้องกันปัญหาการกะเวลาพลาดจนต้องมาตัดพีเจอรันทิ้งเหมือนช่วง Sprint 2 ที่ผ่านมา

วรรณภา ดงงาม – อยากเปลี่ยนวิธีการทำงานของตัวเองคือการพูดคุย อยากพูดคุยกันให้มากขึ้น ไม่เข้าอะไรก็จะทันที ด้วยตัวของตัวเองเป็นคนค่อนข้างเข้าใจยาก จึงอยากเปลี่ยนการพูดคุยของตัวเอง

ทัตพงศ์ ทรายคำ – หากทำงานใหม่อีกครั้ง จะเริ่มจากการใช้เทคนิค TDD ตั้งแต่ต้นเพื่อให้การ test ขึ้นพื้นฐานสำเร็จก่อนการเขียน code จริง เพื่อให้ง่ายต่อการทำงานของฝ่ายอื่นในอนาคต เช่น qa

ปณัฏฐา จันทรศิริ – หากมีโอกาสทำใหม่ จะให้ความสำคัญกับการวางแผนการทดสอบมากขึ้น เช่น การแยกกรณีการใช้งานปกติ กรณีข้อมูลผิดพลาด และ edge case ต่าง ๆ เพื่อช่วยตรวจสอบระบบได้ละเอียดขึ้น รวมถึงช่วยให้ทีมเห็นข้อผิดพลาดได้เร็วตั้งแต่ก่อนนำระบบไปใช้งานจริง

เปรมซ์ เผ่าเสถียรฐานนท์ – หากทำงานใหม่อีกครั้ง จะหาข้อมูล software ที่ใช้งานกับงานให้มากขึ้น และใช้งาน github ให้ถึงศักยภาพของมันให้สูงที่สุด และหาวิธีขึ้น deploy ที่มีประสิทธิภาพและง่ายในการ setup กว่าเดิม

📄 Issue #5: Backend Build ล้มเหลวบน Render

สถานะ: ✅ แก้ไขเรียบร้อยแล้ว (4 commits: 5 เม. - 6 พ.ค. 2026)

Root Cause: Prisma Client ไม่ถูกสร้างก่อน compilation + corepack enable ทำให้เกิด EROFS errors

การแก้ไข:

- สร้าง prisma-generate.cjs พร้อม caching
- ลบ corepack enable ออกจาก build commands
- ระบุ Node.js engine version ($\geq 18.18.0$)
- อัปเดตเอกสาร deployment

📄 FP Analysis: docs/fp-analysis.md (2.5 KB)

สถานะ: ⚠️ ขาด 6 ส่วน (มีเพียง 2 จาก 8 ส่วน)

ขาด: GSC Factors, VAF Calculation, AFP Calculation, Complexity Tables, Sprint Breakdown, Progress Tracking

ความแตกต่าง:

รายการ	ไฟล์ 2.5 KB	PDF เต็มรูปแบบ
UFP	142	174
Total Hours	2,840	1,400
Team Size	6	2

Q: ทีมเป็นกลุ่มเดียวที่ไม่มี GitHub Actions workflows ใช้ Render deploy hook อย่างเดียว — เพียงพอตามมาตรฐาน CLO 2 (ISO/IEC/IEEE 12207) ไหม และตอนเลือก Render แทน GitHub Actions เพราะอะไร

A: สำหรับขั้นพื้นฐาน ถือว่าพอในเชิง deployment process ตาม CLO 2 เพราะมีเอกสาร deployment, มี Render configuration, และมีขั้น build/deploy ที่ทำงานได้จริง แต่ยังไม่พอในเชิง CI เต็มรูปแบบ เพราะยังไม่มี GitHub Actions มารัน test/lint อัตโนมัติก่อน deploy ดังนั้นมันเพียงพอสำหรับ release automation ขั้นต้น แต่ยังเป็นจุดที่ปรับปรุงได้เพื่อให้สอดคล้องกับแนวคิด verification ใน ISO/IEC/IEEE 12207 มากขึ้น และที่เลือก Render ไม่ใช่เพราะมันแทน GitHub Actions ตรงๆ แต่เพราะ Render ช่วยลดภาระ DevOps สำหรับทีมเล็ก deploy ได้เร็ว มี hosting + database + env management ในที่เดียว เหมาะกับ MVP และข้อจำกัดเวลา มากกว่าการตั้ง pipeline และ infrastructure เอง อยู่ใน Stage Deploy ซึ่งเป็นขั้นตอนสุดท้าย

Q: README ระบุว่า Sprint 2 thatphongsai khm รับผิดชอบ 8 งานพัฒนาพร้อมกัน — สมเหตุสมผลหรือเป็นอาการ "งานทุกอย่างกองที่คนเดียว" ทีมแบ่ง/มอบหมายงานอย่างไร?

A: แม้งานใน Sprint 2 จะถูก assign ให้ thatphongsai khm หลายงาน แต่ส่วนใหญ่เป็นงานย่อยและอยู่ในโมดูลเดียวกัน (Frontend Components & Data Integration) จึงให้ดูแลเรื่องเพื่อให้พัฒนาได้เร็วขึ้น โดยทีมยังช่วยกัน review, test และติดตามงานร่วมกัน เพื่อลดความเสี่ยงเรื่อง bottleneck